Web Datamining & Semantics — Final Project Report

Knowledge Graph Construction, Alignment, Reasoning & RAG over RDF/SPARQL

A Neuro-Symbolic Pipeline for Premier League Football

BACHABI Rochdy & SAINT ANDRE Jeffrey

ESILV — A4 DIA, Semester 2 | March 2026

Component	Technology	Key Result
Data Acquisition	football-data.org API	20 teams, 667 players
KB Construction	RDFLib + Wikidata	2,772 → 64,910 triples
Alignment	OWL:sameAs + SequenceMatcher	≥90% confidence threshold
SWRL Reasoning	owlready2 + SPARQL CONSTRUCT	+1,294 inferred triples
KGE	PyKEEN: TransE + ComplEx + DistMult	TransE MRR = 0.1766
RAG Pipeline	Groq llama-3.3-70b + rdflib	0.43 → 1.00 vs baseline

Table 1 — Project overview

GitHub link: https://github.com/LeJ-04/web-datamining-semantics.git

Video link: https://youtu.be/gGVF1trnNGU

1. Data Acquisition & Information Extraction

1.1 Domain & Data Source

This project focuses on the English Premier League football domain. The primary data source is the **football-data.org REST API**, providing structured JSON covering all 20 Premier League teams, full squads, head coaches, and geographical information. The domain's rich relational structure — players belong to teams, teams have coaches, players have nationalities — makes it ideal for KG construction and reasoning.

1.2 Crawler Design & Ethics

Data collection uses a single authenticated API call to *https://api.football-data.org/v4/competitions/PL/teams*. The API key is stored securely via Google Colab Secrets. The football-data.org API is a licensed service permitting research usage; no web scraping was performed and rate limits were respected.

1.3 Cleaning Pipeline

Entity names were normalized by replacing spaces with underscores. The initial graph was checked for isolated triplets — **0 found**. Duplicates are prevented natively by rdflib.

1.4 NER & Ambiguity Cases

Three notable ambiguity cases were encountered:

Case	Entity	Resolution
Name collision	Players sharing surnames (e.g. multiple 'Smith')	Full name URI: ex:Adam_Smith vs ex:John_Smith
Multi-nationality	Players with dual citizenship	Primary nationality from API retained
Position normalization	Raw ('Defence','Midfield') vs fine-grained	Wikidata expansion added fine-grained positions

Table 2 — Ambiguity cases and resolutions

2. KB Construction, Alignment & Expansion

2.1 RDF Modeling Choices

The KB was built using **RDFLib** with two custom namespaces: *http://www.example.org/football/* for entities and *http://www.example.org/football/prop/* for object properties. Six core relations were modeled: playsFor, headCoach, locatedIn, nationality, and playsPosition.

Metric	Initial KB	After Alignment	After SWRL
Triples	2,772	64,910	66,996 (+1,294 inferred)
Entities	775	38,655	38,655
Predicates	6	459	461 (+competesIn, +coachedBy)
Subjects	771	19,382	19,382

Table 3 — KB statistics at each pipeline stage

2.2 Entity Linking with Wikidata

Entity alignment used **Wikidata** as the reference OKB. Strategy: batch alignment of players by team using **SequenceMatcher** (difflib) with a **90% similarity threshold**, followed by individual alignment for remaining entities. Successful matches were recorded as **OWL:sameAs** triples, expanding the graph from 2,772 to **64,910 triples** (459 distinct predicates).

2.3 SPARQL Expansion Strategy

SPARQL CONSTRUCT queries pulled additional Wikidata properties for aligned entities. Federated queries used exponential backoff retry logic (up to 3 attempts, 1.5x multiplier).

3. Reasoning & Knowledge Graph Embeddings

3.1 SWRL Reasoning

Two **SWRL rules** were declared using owlready2 to infer hidden knowledge. Pellet (native SWRL reasoner) requires Java 25+ (Colab has Java 21), so rules were executed via **SPARQL CONSTRUCT** — semantically equivalent:

Rule	SWRL Logic	New Property	Triples Added
coachedBy	Person(?p) ∧ Team(?t) ∧ Person(?c) ∧ playsFor(?p,?t) ∧ headCoach(?t,?c) → coachedBy(?p,?c)	Person → Person	647
competesIn	Person(?p) ∧ Team(?t) ∧ Country(?c) ∧ playsFor(?p,?t) ∧ locatedIn(?t,?c) → competesIn(?p,?c)	Person → Country	647

Table 4 — SWRL rules and inferred triples

Total augmented graph: **66,996 triples** (+1,294 inferred). These properties are critical for RAG: they do not exist in any external dataset or LLM training data — only the augmented KG can answer related questions.

3.2 KGE Data Preparation

The pipeline filtered out literals and blank nodes, retaining only URI→URI→URI triples. A **60/20/20 split** was applied with OOV prevention (unseen entities moved back to train):

Split	Triples	%
Train	52,286	60%
Validation	6,225	20%
Test	6,399	20%
Total	64,910	100%

Table 5 — KGE dataset splits (60/20/20)

3.3 KGE Model Training

Three models were trained using **PyKEEN** on a Google Colab T4 GPU:

Model	Scoring Approach	Loss Function	Time
TransE	Translational: h + r ≈ t in vector space	MarginRankingLoss	318s
ComplEx	Bilinear in complex space — handles asymmetric relations	SoftplusLoss	604s
DistMult	Bilinear diagonal — specializes in symmetric patterns	SoftplusLoss	~400s

Table 6 — KGE model configurations (embedding_dim=128, epochs=200, lr=0.001)

3.4 Evaluation Metrics

Model	MRR	Hits@1	Hits@3	Hits@10
TransE (best)	0.1766	0.0987	0.1946	0.3526
ComplEx	0.0244	0.0099	0.0206	0.0498
DistMult	0.0214	0.0083	0.0190	0.0449

Table 7 — KGE test set metrics (both directions combined)

TransE outperforms both ComplEx and DistMult. The graph is very sparse (~1.5 triples/entity) and dominated by asymmetric relations (playsFor), where TransE's translational approach is the best fit. ComplEx and DistMult require denser, more varied graphs to show their advantages.

3.5 Head vs Tail Prediction

Model	Head MRR	Tail MRR	Why tail is easier
TransE	0.1236	0.2296	Tail = pick 1 of 20 teams; Head = pick 1 of 667 players
DistMult	0.0154	0.0275	Same asymmetry, lower overall performance
ComplEx	—	—	Performs equally poorly in both directions

Table 8 — Head vs tail prediction breakdown

3.6 Size Sensitivity Analysis

Models were re-trained at 20k / 50k / full (50 epochs) to measure data scaling:

Model	20k MRR	50k MRR	Full MRR	Scaling trend
TransE	0.0256	0.0634	0.0987	Steady linear growth — reliable at any size
ComplEx	0.0003	0.0095	0.0244	Near-zero until 50k; needs critical mass
DistMult	0.0012	0.0089	0.0214	Strong jump from 50k to full data

Table 9 — MRR sensitivity to training set size (50 epochs)

TransE is the most data-efficient model. ComplEx and DistMult require a critical data mass before learning meaningful patterns, suggesting they would benefit from either more training data or dedicated hyperparameter tuning.

3.7 t-SNE Embedding Visualization

t-SNE projected the 128D embeddings to 2D (ComplEx excluded — its complex-valued representations cannot be directly projected). The two available models show fundamentally different structures:

Model	Visual pattern	Interpretation
TransE	Loose cloud — categories distinct but overlapping	Preserves relational nuance. Entities connected by shared relations stay near each other. Better 'roadmap' of how the graph is organized.
DistMult	Tight dense islands — hyper-specific clusters	Forces same-type entities into near-identical positions. All PL teams compress to near-single point. Superior categorical separation but loses inter-category nuance.

Table 10 — t-SNE visualization comparison: TransE vs DistMult

Nearest-neighbor validation confirms both models learned meaningful semantics: Arsenal's nearest neighbors are all Premier League clubs, England's are European nations, and Mohamed Salah clusters with international attackers. TransE provides a better relational roadmap; DistMult provides a better categorical map.

4. RAG over RDF/SPARQL

4.1 Schema Summary

Injecting 66,996 triples into the LLM context is impossible. Instead, a **Schema Summary** (~30 lines) is injected into every prompt, covering: class hierarchy with instance counts, object properties with usage stats, entity naming conventions (underscores), and SPARQL generation rules.

Class	Instances	Key Properties
ex:Person	667	playsFor, nationality, playsPosition, coachedBy, competesIn
ex:Country	68	— (target of nationality, locatedIn, competesIn)
ex:Team	20	headCoach, locatedIn
ex:FootballPosition	16	— (target of playsPosition)

Table 11 — Schema Summary: classes injected into every LLM prompt

4.2 NL to SPARQL Prompt & LLM

The prompt combines the Schema Summary with strict instructions: return only raw SPARQL, use LIMIT 20 by default, apply FILTER/CONTAINS/LCASE for flexible matching, temperature=0.1 for determinism. LLM: **Groq API — llama-3.3-70b-versatile** (free tier, no credit card, 30 req/min).

4.3 Self-Repair Mechanism

When a query fails (syntax error or 0 results), the failed query + error are sent back to the LLM with a targeted repair prompt (up to 3 attempts). Demonstrated: 'Who coaches Chelsea FC players?' — repaired in 1 attempt, returning **Liam Rosenior** via the SWRL-inferred prop:coachedBy.

4.4 Baseline vs RAG Evaluation

7 questions evaluated (baseline = same LLM, no KG access):

ID	Category	Baseline	RAG	Key Insight
Q1	Simple lookup	0.5	1.0	RAG has exact KG roster
Q2	Aggregation	0.5	1.0	RAG: exact count (246 English players)
Q3	SWRL inferred	0.0	1.0	Baseline cannot use competesIn
Q4	Simple lookup	0.5	1.0	RAG: complete 20-team list
Q5	SWRL inferred	0.0	1.0	Baseline: no coachedBy knowledge
Q6	Filtered lookup	0.5	1.0	RAG: exact 41 French players
Q7	Direct lookup	1.0	1.0	Both know Arne Slot (well-known fact)
Avg	—	0.43	1.00	+0.57 overall improvement

Table 12 — Baseline vs RAG evaluation (0=incorrect, 0.5=partial, 1.0=correct)

Q3 and Q5 score **0.0 for the baseline** but 1.0 for RAG — these involve SWRL-inferred properties that exist only in the augmented KG, demonstrating the value of the Neuro-Symbolic approach.

4.5 Interactive Demo

5 demo questions — all resolved on first attempt:

Question	Answer	Property Used
Players for Liverpool FC?	20 players (Alisson, Salah...)	prop:playsFor
Players competing in England?	647 players	prop:competesIn (SWRL)
Spanish nationality players?	20 players (Raya, Cucurella...)	prop:nationality
Head coach of Man City FC?	Pep Guardiola	prop:headCoach
Question	**Answer**	**Property Used**
Players coached by Arne Slot?	20 players	prop:coachedBy (SWRL)

Table 13 — Interactive demo results (D.5)

5. Critical Reflection

5.1 Knowledge Base Quality Impact

The football-data.org API provided clean structured data, explaining the near-100% SPARQL execution rate. However, Wikidata alignment introduced 459 predicates — most irrelevant to football — adding noise to KGE training. Selective alignment (football-relevant properties only) would improve KGE metrics.

5.2 Noise Issues

Two main noise sources: (1) ~62,000 Wikidata triples with sparse, irrelevant predicates; (2) raw Wikidata Q-IDs appearing in nearest-neighbor results — traces of incomplete entity resolution after alignment.

5.3 Rule-Based vs Embedding-Based Reasoning

SWRL rules are precise and interpretable but require explicit design and cannot generalize. KGE models discover latent patterns for link prediction but are probabilistic. The pipeline combines both: SWRL facts are precisely queryable via RAG, while KGE embeddings could support fuzzy similarity retrieval.

5.4 What We Would Improve

Area	Current Limitation	Proposed Improvement
KGE split	60/20/20 leaves less training data	Return to 80/10/10 — more training triples improve embeddings
KGE models	ComplEx and DistMult underperform on sparse graph	Tune lr and regularization; try RotatE for antisymmetric relations
Alignment	459 Wikidata predicates — mostly noisy	Whitelist football-relevant properties only (P106, P641, P1532...)
SWRL	Pellet requires Java 25+ — unavailable in Colab	Upgrade Java or use HermiT with DL-safe rules
RAG	Prompt-based SPARQL generation only	Add KGE-based vector similarity for fuzzy retrieval
Evaluation	Manual scoring for baseline comparison	Automated F1 / exact-match with gold SPARQL annotations

Table 14 — Limitations and proposed improvements

6. Conclusion

This project successfully implemented a complete Neuro-Symbolic pipeline for the Premier League football domain, integrating KB construction, ontology alignment, symbolic SWRL reasoning, knowledge graph embeddings, and retrieval-augmented generation.

Among three KGE models, **TransE proved the most robust** (MRR=0.1766, Hits@10=0.3526) on this sparse, asymmetric-relation graph. The t-SNE comparison revealed a fundamental trade-off: TransE preserves relational nuance while DistMult maximizes categorical clarity.

The SWRL-inferred properties — invisible to any LLM's training data — were reliably queried through the RAG system, achieving a **+0.57 improvement over the baseline** (0.43 → 1.00 average score). The self-repair mechanism recovered all failed queries within 1-2 attempts, making the pipeline robust in practice.

References

[1] Bordes et al. (2013). Translating Embeddings for Modeling Multi-relational Data (TransE). NeurIPS.

[2] Trouillon et al. (2016). Complex Embeddings for Simple Link Prediction (ComplEx). ICML.

[3] Yang et al. (2015). Embedding Entities and Relations for Learning and Inference in KBs (DistMult). ICLR.

[4] Ali et al. (2021). PyKEEN 1.0: A Python Library for KGE Training and Evaluation. JMLR.

[5] football-data.org API. https://www.football-data.org/

[6] Wikidata SPARQL Endpoint. https://query.wikidata.org/

[7] Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP. NeurIPS.

[8] Groq API Documentation. https://console.groq.com/docs/